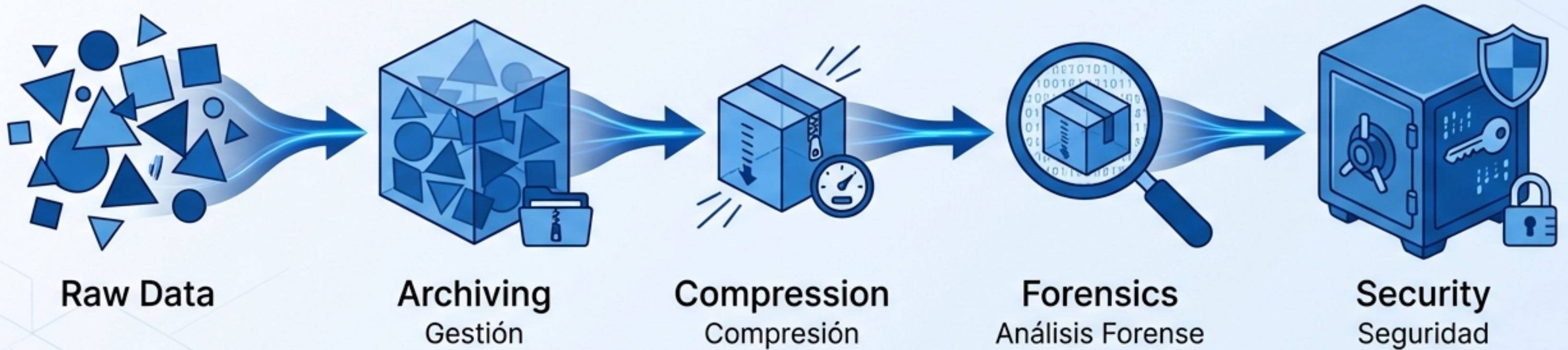


El Ciclo de Vida del Dato en Linux

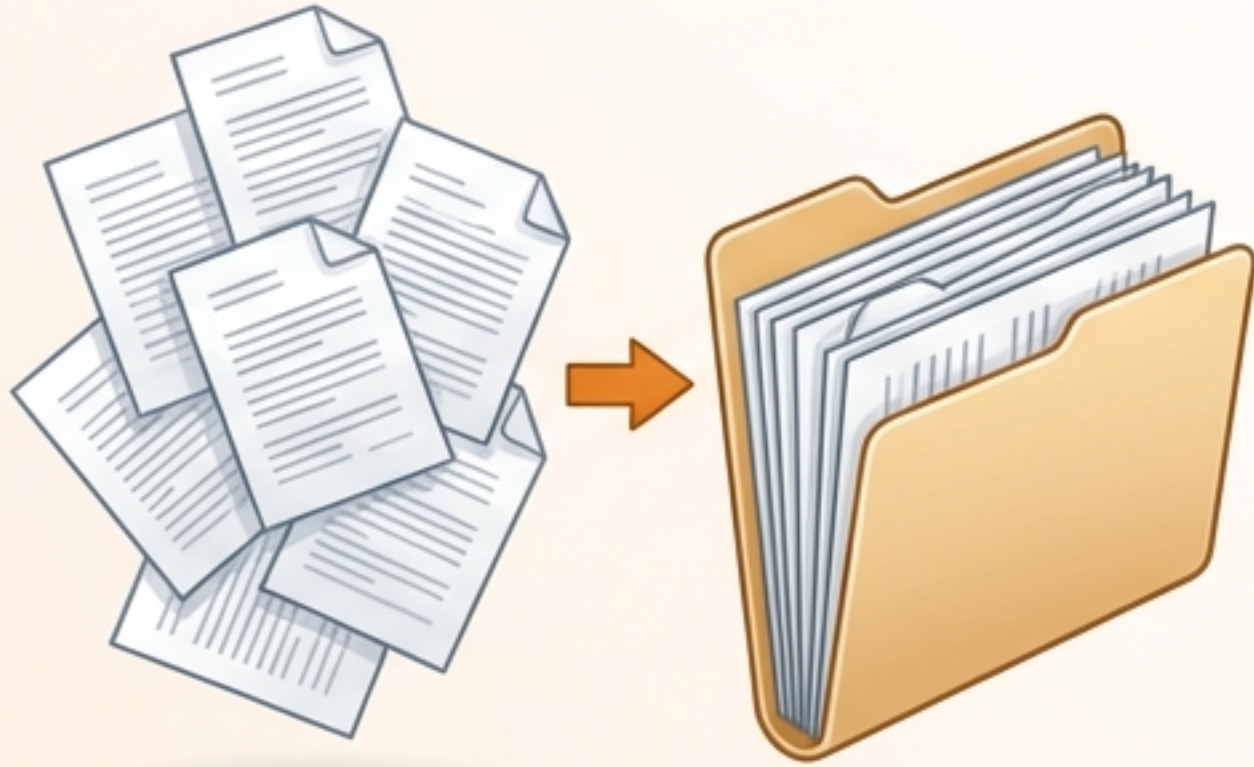
Gestión, Análisis y Seguridad



Una guía técnica para administradores de sistemas: Desde tar y gzip hasta el análisis forense de binarios ELF y la manipulación con dd.

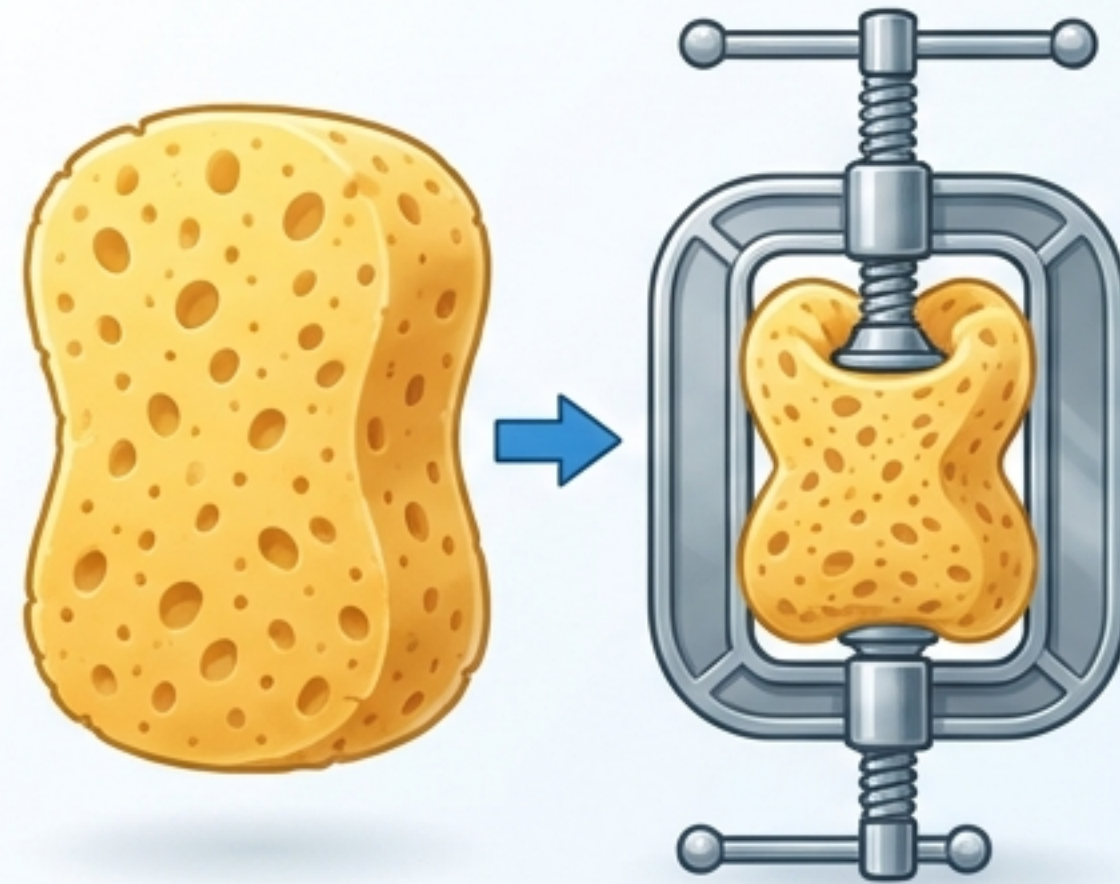
Agrupar vs. Reducir: Entendiendo la Diferencia

Archivar (tar)



- **Combina** múltiples **archivos** en un solo "tarball".
- **No reduce el tamaño** por sí mismo.
- **Analogía:** Organizar documentos en una caja.

Comprimir (gzip, bzip2, xz)



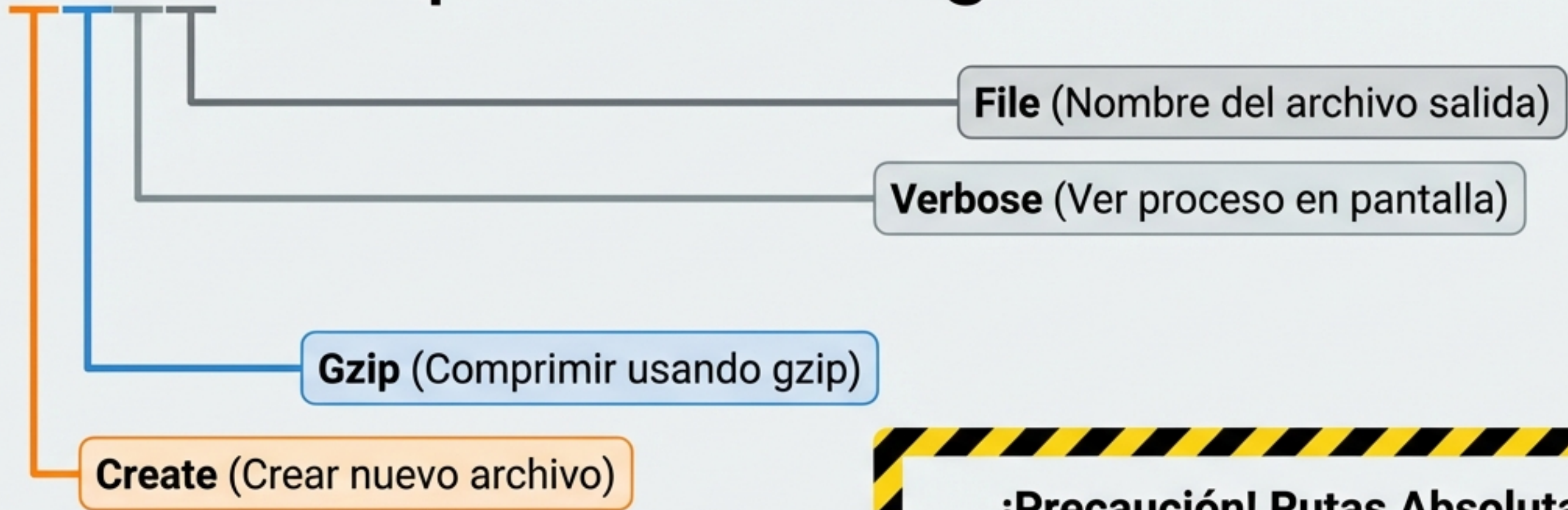
- **Reduce el tamaño** mediante algoritmos matemáticos.
- **Trabaja** sobre un solo archivo a la vez.
- **Analogía:** Eliminar el aire para ahorrar espacio.

En Linux, **combinamos ambos** pasos para crear **archivos .tar.gz**

`tar`: El Archivador Universal

(Tape ARchiver) - La herramienta estándar para backups.

```
tar -czvf respaldo.tar.gz Documentos/
```



¡Precaución! Rutas Absolutas

`tar` elimina la barra inicial `/` (ej: `/home` se vuelve `home`) por seguridad. Esto evita sobrescribir archivos del sistema al descomprimir.

Para extraer: ``tar` -xzvf respaldo.tar.gz``

Compresión Estándar con `gzip`



Uso común:
Rotación de logs y reducción rápida de archivos de texto.



El original es eliminado por defecto.

Tip: Conservar el original

```
gzip -k notas.txt
```

El flag `-k` (Keep) genera el comprimido sin borrar la fuente.

Eficiencia vs. Velocidad

Alternativas de alta compresión

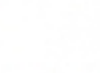
gzip

-  **Velocidad:** Alta
-  **Compresión:** Estándar
-  **Uso:** Tareas diarias, rapidez.
-  **Extensión:** .gz

bzip2

-  **Velocidad:** Media
-  **Compresión:** Alta
-  **Uso:** Distribución de software.
-  **Extensión:** .bz2
-  **Comando:** bunzip2

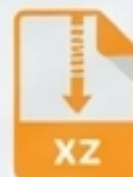
xz (Estándar Moderno)

-  **Velocidad:** Baja
(Intensivo en CPU)
-  **Compresión:** Máxima
-  **Uso:** Archivero a largo plazo.
-  **Extensión:** .xz
-  **Comando:** unxz



gzip: 7.5K

vs →



xz: 6.8K

Interoperabilidad: El Estándar `zip`

El formato ideal para compartir datos con usuarios de Windows o macOS.



```
zip -r -e datos.zip Trabajo/
```

Recursive: Incluye subcarpetas.

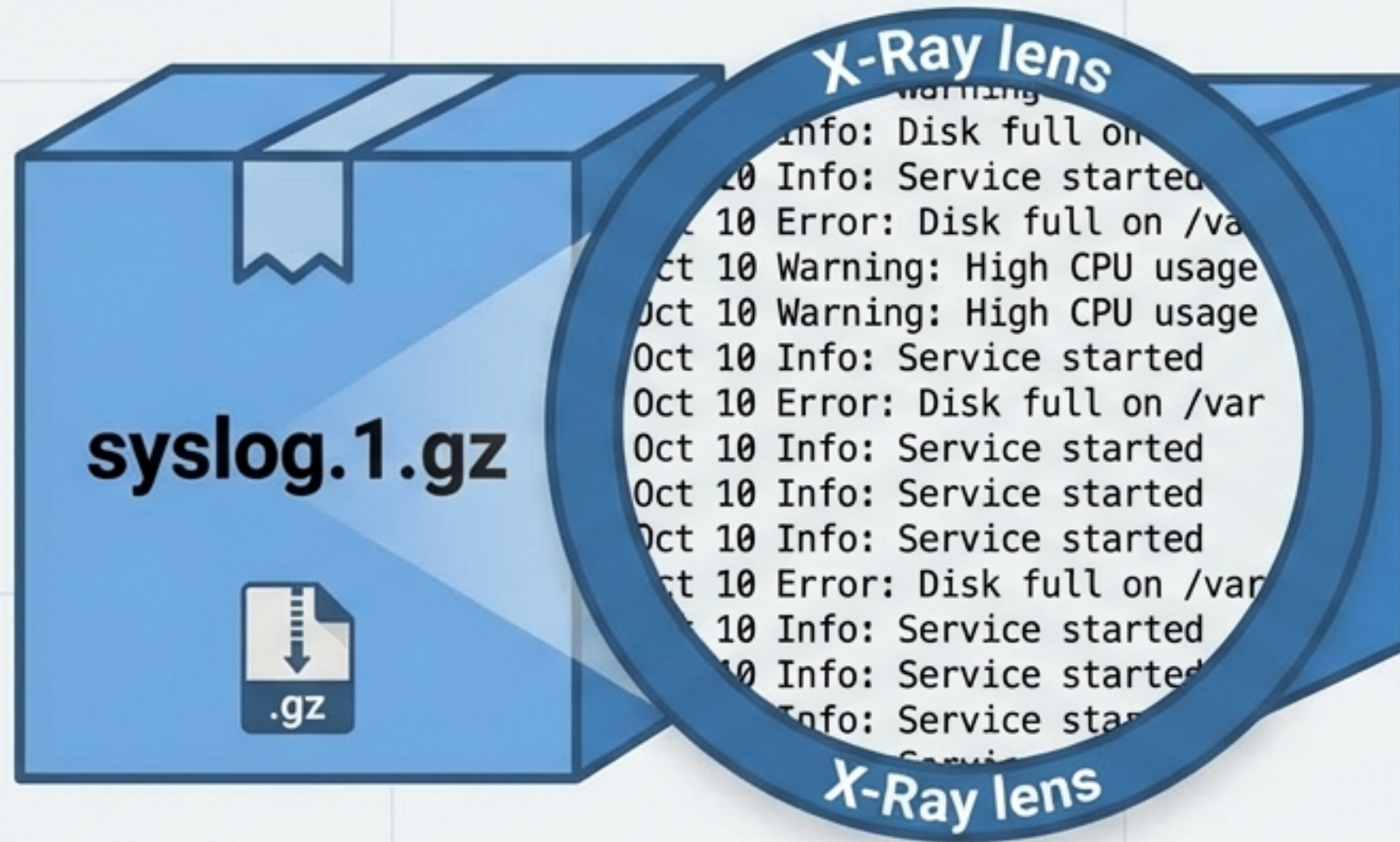
Encrypt: Protege con contraseña.

```
unzip datos.zip -d /tmp/destino
```

Directory: Define dónde descomprimir.

Visión de Rayos X: `zcat`

Inspección de contenido sin extracción.



Caso de uso: Auditoría de Logs

Los sistemas Linux rotan y comprimen logs automáticamente para ahorrar espacio. `zcat` permite leerlos instantáneamente sin llenar el disco descomprimiendo archivos gigantes.

```
zcat /var/log/syslog.1.gz | less
```


Identificación Forense: Más allá de la Extensión

En Linux, la extensión (.txt, .pdf) es irrelevante.



backup.tar.bz2

Comando ``file``



```
backup.tar.bz2: bzip2 compressed  
data, block size 900k
```

```
file /bin/bash  
ELF 64-bit LSB executable, x86-64...
```

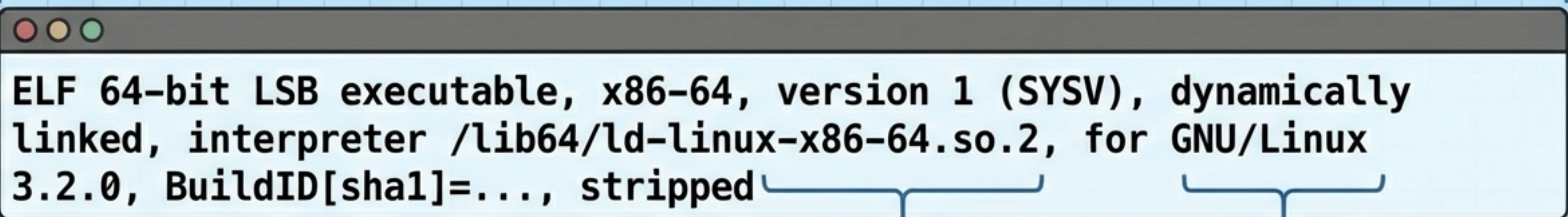


El comando lee los '**Magic Bytes**' (cabecera del archivo) para revelar su verdadera naturaleza.

Anatomía de un Ejecutable: ELF

Formato Binario Estándar (Linux)
para procesadores de 64 bits.

Depende de librerías externas
(.so) para ahorrar espacio.



```
ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically  
linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux  
3.2.0, BuildID[sha1]=..., stripped
```

ELF 64-bit LSB
Estándar (Linux)
para procesadores de
64 bits.

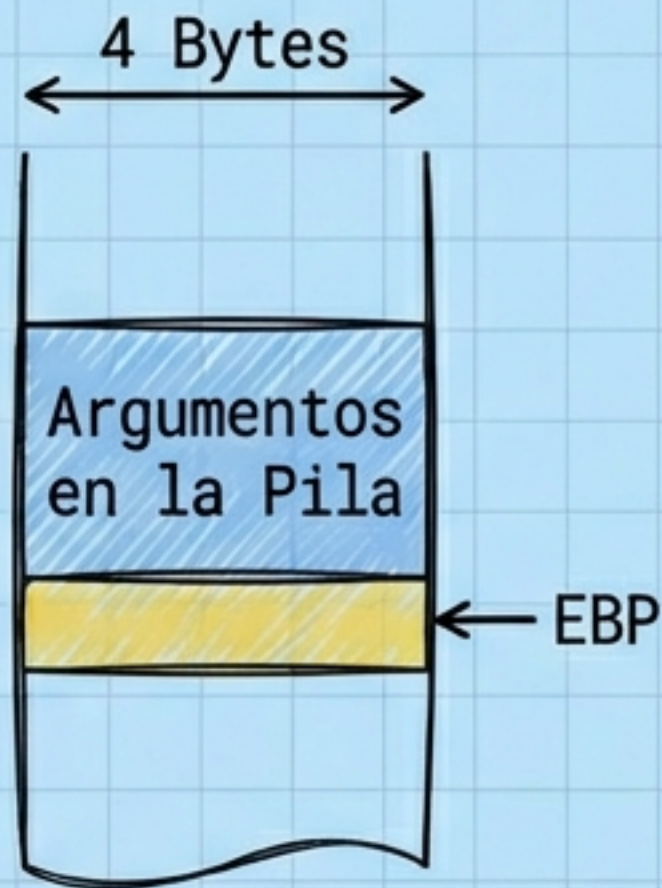
El cargador que
prepara el programa
en memoria.

Sin símbolos de depuración
(Hardening / Menor tamaño).

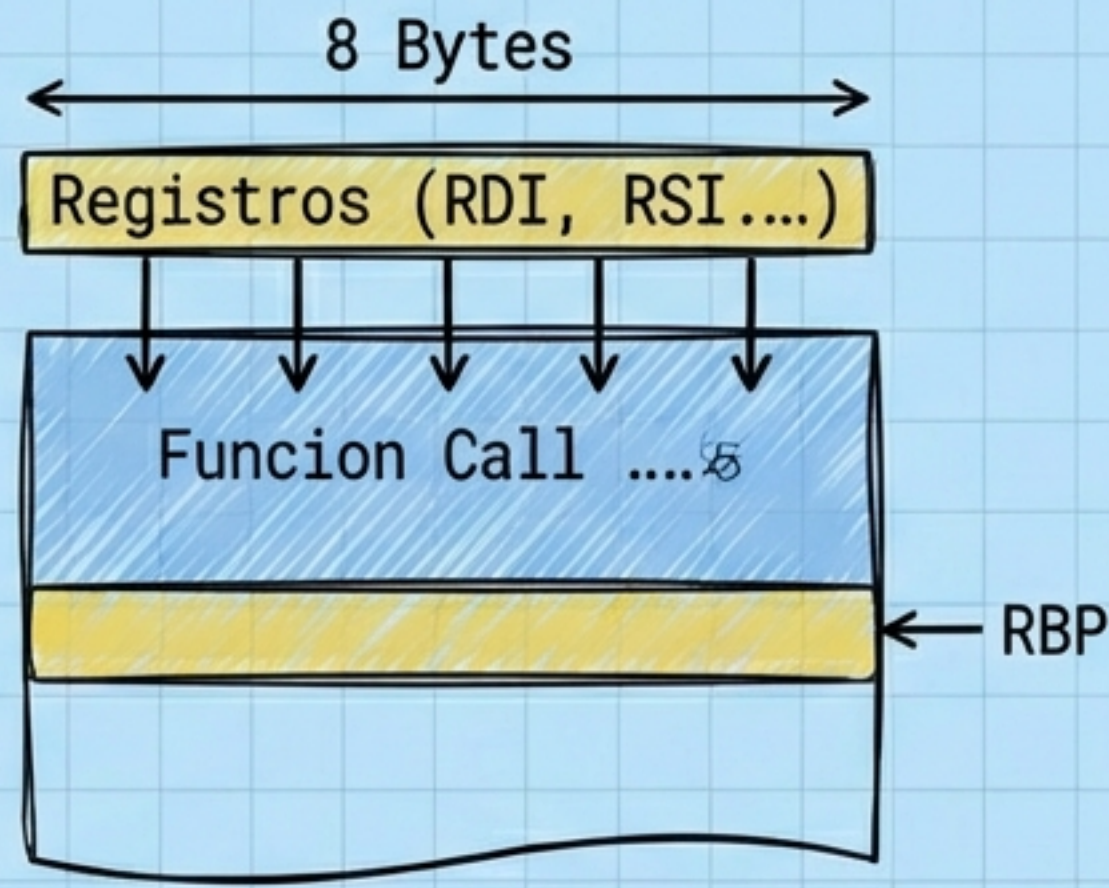
Entender esta salida es vital
para ingeniería inversa y
análisis de seguridad.

Arquitectura: 32-bit vs 64-bit

32-bit (x86)



64-bit (x86-64)

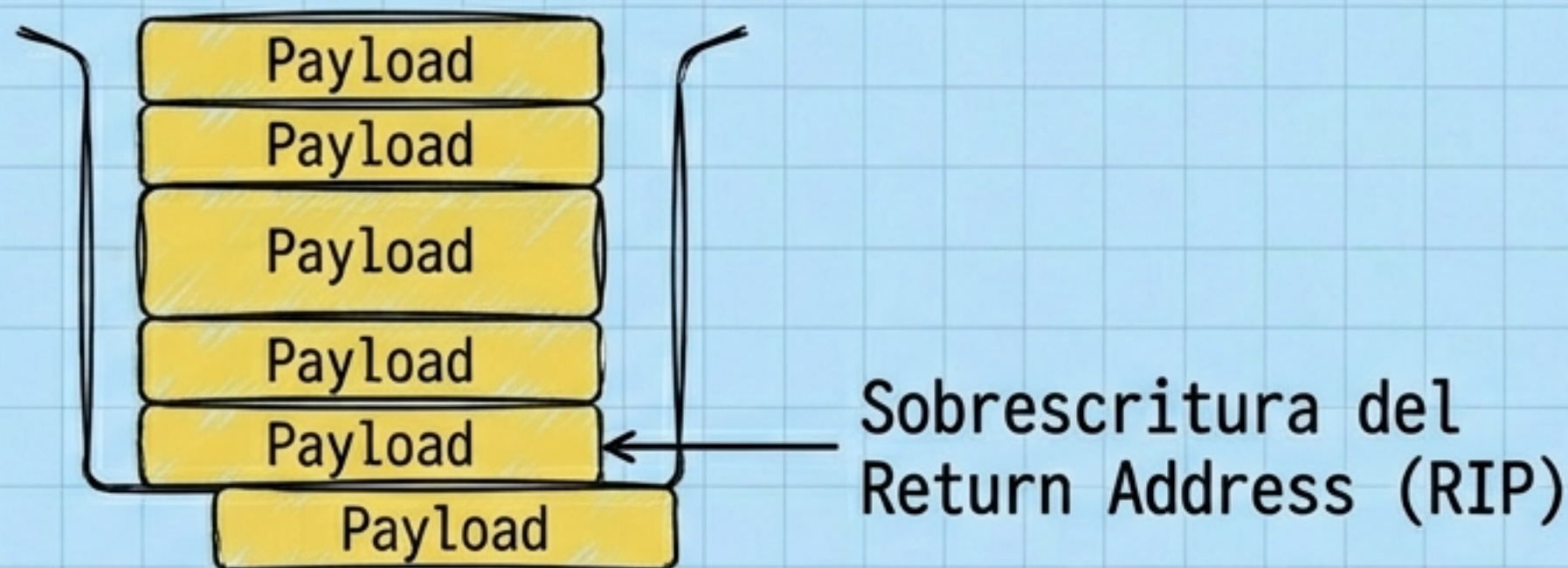


Diferencias Clave:

- **Punteros:** 4 bytes vs 8 bytes.
- **Argumentos:** En la Pila (32-bit) vs. En Registros (64-bit).
- **Registros:** EBP vs RBP.

Impacto en Ciberseguridad: Buffer Overflows

Cómo la arquitectura cambia las reglas de la explotación.



En 64-bit, necesitas 8 bytes para controlar el RIP.
El payload debe ser más grande y preciso.

ROP Chains

Debido a protecciones como NX (No-Execute), los atacantes deben encadenar fragmentos de código existente (Gadgets).

Conclusión

El comando ``file`` es el primer paso en el análisis de vulnerabilidad.

Operaciones Críticas: `dd`

Manipulación a Bajo Nivel (Data Duplicator / Disk Destroyer)

```
dd if=/dev/sda of=/dev/sdb bs=4M status=progress
```

- **Input File:** Origen (Disco, partición o archivo).
- **Output File:** Destino (¡Cuidado! Sobrescribe sin preguntar).
- **Block Size:** Tamaño de lectura/escritura (Optimización).

Copia bit a bit. Ignora el sistema de archivos.
Copia incluso el espacio vacío y archivos borrados.



Casos de Uso y Peligros de `dd` Casos de Uso y Peligros de `dd`

Generar Archivos de Prueba

```
dd if=/dev/zero of=test.dat  
bs=1M count=100
```

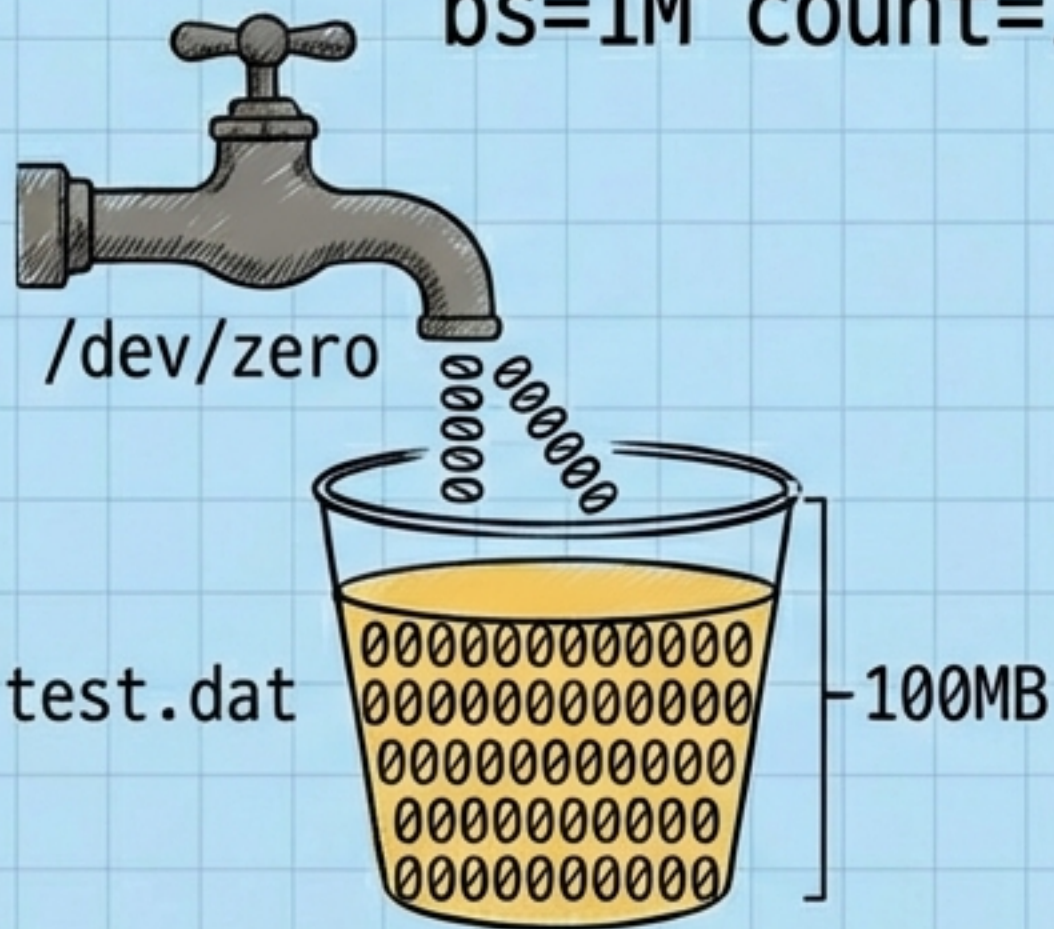


Imagen de Disco Completa

```
sudo dd if=/dev/sda  
of=backup.img ...
```

¡PELIGRO! `dd` no pide confirmación.
Equivocar la letra del disco en `of=`
(ej. sda en vez de sdb) destruirá tu
sistema operativo irreversiblemente.

Hoja de Ruta: Comandos Esenciales

Gestión (Archivar/Comprimir)

``tar -czvf``: Crear .tar.gz
``tar -xzvf``: Extraer .tar.gz
``gzip -k``: Comprimir
(mantener original)

Optimización

``zip -r -e``: Comprimir
carpeta con password
``xz``: Máxima compresión
(lento)

Análisis Forense

``zcat``: Leer sin descomprimir
``file``: Identificar Magic
Bytes / Arquitectura

Operaciones Críticas

``dd``: Clonado bit a bit
Recordatorio: Verificar
siempre ``of=`` (Output File).